

Chapter 48: Randomized Algorithms

Personal Notes

Most algorithms are DETERMINISTIC — same input, same output, every time. Randomized algorithms deliberately use random numbers to make choices, and in return they get simpler code, better average-case performance, or clean solutions to problems that seem to require impossible feats. Quickselect finds the k th element in $O(n)$ AVERAGE time without sorting. Reservoir sampling picks a uniform random element from a stream you can only read once. Fisher-Yates shuffles an array in $O(n)$ with a beautifully simple invariant.

These aren't just theoretical curiosities — they're the algorithms powering database engines, streaming systems, and A/B testing platforms. And they show up in Google interviews often enough to be worth memorizing. This chapter covers the three big ones (Quickselect, Reservoir Sampling, Fisher-Yates), plus the Monte Carlo / Las Vegas classification and a handful of interview-favorite problems built on them.

1. Why Randomize?

Motivation	Where It Helps
Sorting to find a rank	Sort is $O(n \log n)$. Quickselect is $O(n)$ average.
Streaming data	Can't fit stream in memory — reservoir sampling picks fairly with $O(1)$ space.
Adversarial inputs	Randomization prevents worst-case exploits (e.g., in quicksort).
Simpler algorithm	Randomized primality tests are far simpler than deterministic ones.
Load balancing	Random assignment gives probabilistic uniformity — no coordination needed.

2. Two Kinds — Las Vegas vs Monte Carlo

Randomized algorithms come in two flavors. Knowing the difference matters when analyzing them:

Type	Guarantees Correctness?	Guarantees Runtime?
Las Vegas	ALWAYS correct	Random time (fast in expectation)
Monte Carlo	Fast time	Possibly wrong with small probability

- Quickselect, Randomized Quicksort → Las Vegas (always correct, average $O(n)$)

- Miller-Rabin primality test → Monte Carlo (occasionally wrong, always fast)
- Reservoir sampling → Las Vegas (always fair, deterministic runtime)

For interviews: Almost every randomized algorithm you'll code in an interview is Las Vegas — the answer is right, but the runtime is expected $O(\dots)$ rather than worst-case. Explaining this out loud (" $O(n)$ average, $O(n^2)$ worst case for random inputs, but the randomization prevents adversarial worst cases") signals mastery.

3. Quickselect — Kth Smallest in $O(n)$ Average

Quickselect finds the k th smallest element in an unsorted array WITHOUT sorting the whole thing. Same partitioning idea as quicksort — but after each partition, we only recurse into ONE side (the side containing rank k). This drops the total work from $O(n \log n)$ to $O(n)$ on average.

The Idea

```
arr = [3, 2, 1, 5, 6, 4], k = 2 (2nd smallest)

Pick a pivot (say arr[5] = 4). Partition:
  Elements < 4:  [3, 2, 1]           positions 0..2
  Pivot 4:       position 3
  Elements > 4:  [5, 6]             positions 4..5

Pivot is at position 3.
Target rank k-1 = 1 (2nd smallest = index 1 in sorted order).

1 < 3 → recurse into left partition [3, 2, 1].

Pick another pivot, partition, recurse.
Eventually find that the 2nd smallest is 2.
```

Full Code

```
public int quickSelect(int[] nums, int k) {
    // Returns the (k-1)-th index in sorted order
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int pivotIdx = partition(nums, lo, hi);
        if (pivotIdx == k - 1) return nums[pivotIdx];
        else if (pivotIdx < k - 1) lo = pivotIdx + 1;
        else hi = pivotIdx - 1;
    }
    return -1; // shouldn't happen for valid k
}

private int partition(int[] nums, int lo, int hi) {
    // Randomized pivot – protects against adversarial inputs
    int randIdx = lo + (int) (Math.random() * (hi - lo + 1));
```

```

    swap(nums, randIdx, hi);

    int pivot = nums[hi];
    int i = lo;
    for (int j = lo; j < hi; j++) {
        if (nums[j] <= pivot) {
            swap(nums, i, j);
            i++;
        }
    }
    swap(nums, i, hi);
    return i;
}

private void swap(int[] a, int i, int j) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}

// Time: O(n) average, O(n²) worst case (rare with random pivot)
// Space: O(1) extra (iterative)

```

Why O(n) Average

Each partition step scans the current range: $O(\text{range})$ work.
A random pivot splits the range into roughly half on average.

Total work: $n + n/2 + n/4 + n/8 + \dots = 2n = O(n)$

Compare to quicksort, which processes BOTH halves after each split:
 $n + 2*(n/2) + 4*(n/4) + \dots = O(n \log n)$

The random pivot is essential: Without randomization, an adversary could feed sorted input and force $O(n^2)$ worst case. Random pivots make the algorithm's expected complexity $O(n)$ regardless of input pattern. For interviews, always use `Math.random()` or `ThreadLocalRandom` for the pivot — it's the difference between passing and TLE on adversarial test cases.

4. Reservoir Sampling — Sampling from a Stream

Given a STREAM of unknown length (numbers arriving one at a time, can't store them all), pick ONE element uniformly at random. Reservoir sampling does this in $O(1)$ memory with a beautiful invariant: after processing i elements, each has probability $1/i$ of being the current pick.

The Algorithm

1. Keep the current pick in a variable, initialized to the first element
2. For the i -th element (1-indexed), replace the pick with probability $1/i$
3. After processing all n elements, the pick is uniformly random

```

public class ReservoirSample {
    Random rand = new Random();
    int pick;
    int count = 0;

    public void offer(int value) {
        count++;
        // Replace pick with probability 1/count
        if (rand.nextInt(count) == 0) {
            pick = value;
        }
    }

    public int getPick() {
        return pick;
    }
}

// Time: O(1) per element | Space: O(1)

```

Why It's Fair — Induction Argument

After processing i elements, each has probability $1/i$ of being the pick.

Base case ($i=1$): First element is always picked. Probability = $1/1 = 1$ ✓

Inductive step: Assume true for i . When processing element $(i+1)$:

- New element becomes pick with probability $1/(i+1)$ ✓
- Old pick survives with probability $(i/(i+1))$ – that's 1 minus $1/(i+1)$
- Any previously-tracked element was picked with $1/i$, now it survives with probability $(1/i) \times (i/(i+1)) = 1/(i+1)$ ✓

All elements – old and new – have the same $1/(i+1)$ probability. By induction, this holds for any stream length.

The magic of $1/i$: `rand.nextInt(count)` returns a value in $[0, \text{count})$. The chance it's 0 is exactly $1/\text{count}$. That single line — replace the pick with probability $1/i$ — turns out to give exactly uniform sampling. Elegant math wrapped in one comparison.

5. Reservoir Sampling for K Elements

Sampling ONE element is neat. What about K? Same idea, slightly generalized: maintain a reservoir of size K. For the i -th element ($i > K$), replace a random slot in the reservoir with probability K/i .

```
public class KReservoir {
    int[] reservoir;
    int k, count = 0;
    Random rand = new Random();

    public KReservoir(int k) {
        this.k = k;
        this.reservoir = new int[k];
    }

    public void offer(int value) {
        count++;
        if (count <= k) {
            reservoir[count - 1] = value;    // fill initial reservoir
        } else {
            int j = rand.nextInt(count);    // uniform in [0, count)
            if (j < k) reservoir[j] = value;
        }
    }

    public int[] getReservoir() {
        return reservoir;
    }
}

// Time: O(1) per element
// Space: O(k)
```

At every point, the reservoir contains a uniform random sample of size K from all elements seen so far. Used in log processing, database sampling, and streaming analytics.

6. Fisher-Yates Shuffle — Uniform Random Permutation

How do you shuffle an array so every one of $n!$ permutations is equally likely? FISHER-YATES (also called Knuth Shuffle) does it in $O(n)$ with a beautifully simple loop.

```
public void shuffle(int[] arr) {
    Random rand = new Random();
    for (int i = arr.length - 1; i > 0; i--) {
        int j = rand.nextInt(i + 1);    // random index in [0, i]
        int tmp = arr[i];
        arr[i] = arr[j];
        arr[j] = tmp;
    }
}
```

```
}
```

```
// Time: O(n) | Space: O(1)
```

Why It's Fair

Position n-1: pick one of n elements → probability $1/n$ each
Position n-2: pick one of remaining n-1 elements → probability $1/(n-1)$

...

Position 0: only one choice left → probability 1

Total probability of any specific permutation:

$$1/n \times 1/(n-1) \times \dots \times 1/1 = 1/n!$$

Every permutation is equally likely. Perfect uniformity.

The naive shuffle is BROKEN: A common mistake: for each i , swap $\text{arr}[i]$ with a random index in $[0, n-1]$. This gives n^n possible outcome sequences, but there are only $n!$ permutations. Not all permutations end up equally likely — some appear more often. Fisher-Yates restricts the random index to $[0, i]$ to guarantee uniformity.

7. Weighted Random Sampling

Sometimes elements have DIFFERENT probabilities of being picked. Given weights $[1, 3, 2]$, you want element 0 with probability $1/6$, element 1 with $3/6$, element 2 with $2/6$. Solved with PREFIX SUMS + BINARY SEARCH — a classic Google interview problem.

```
class WeightedRandom {
    int[] prefixSums;
    int total;
    Random rand = new Random();

    public WeightedRandom(int[] weights) {
        prefixSums = new int[weights.length];
        int sum = 0;
        for (int i = 0; i < weights.length; i++) {
            sum += weights[i];
            prefixSums[i] = sum;
        }
        total = sum;
    }

    public int pickIndex() {
        int target = rand.nextInt(total) + 1; // 1..total
        // Binary search for smallest prefix ≥ target
        int lo = 0, hi = prefixSums.length - 1;
        while (lo < hi) {
```

```

        int mid = lo + (hi - lo) / 2;
        if (prefixSums[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// pickIndex: O(log n)
// LeetCode 528: Random Pick with Weight

```

How It Works

```

weights: [1, 3, 2]
prefixSums: [1, 4, 6]  total = 6

Random target in 1..6:
    target = 1          → prefix ≥ 1 → index 0 (weight 1)
    target = 2, 3, 4    → prefix ≥ 2/3/4 → index 1 (weight 3)
    target = 5, 6       → prefix ≥ 5/6 → index 2 (weight 2)

Probabilities: index 0 = 1/6, index 1 = 3/6, index 2 = 2/6. Correct.

```

8. Classic Problem 1 — Kth Largest Element in an Array

LeetCode 215. Find the kth LARGEST element in an unsorted array. Three viable approaches:

- Sort — $O(n \log n)$ time. Simple but wasteful.
- Min-heap of size k (Ch 34) — $O(n \log k)$. Better for small k .
- Quickselect — $O(n)$ average. Best when time is critical.

```

public int findKthLargest(int[] nums, int k) {
    // Kth largest = (n - k)th smallest (0-indexed)
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}

private int quickSelect(int[] nums, int lo, int hi, int target) {
    if (lo == hi) return nums[lo];

    int pivotIdx = partition(nums, lo, hi);
    if (pivotIdx == target) return nums[pivotIdx];
    else if (pivotIdx < target) return quickSelect(nums, pivotIdx + 1, hi, target);
    else return quickSelect(nums, lo, pivotIdx - 1, target);
}

private int partition(int[] nums, int lo, int hi) {
    int r = lo + (int)(Math.random() * (hi - lo + 1));
    swap(nums, r, hi);
}

```

```

    int pivot = nums[hi];
    int i = lo;
    for (int j = lo; j < hi; j++) {
        if (nums[j] < pivot) { swap(nums, i, j); i++; }
    }
    swap(nums, i, hi);
    return i;
}

private void swap(int[] a, int i, int j) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}

// Time: O(n) average, O(n^2) worst case

```

When to use which: For Kth Largest: use heap (Ch 34) if you want guaranteed $O(n \log k)$. Use Quickselect if the interviewer specifically asks for $O(n)$ average. In practice, heap is more reliable in production; Quickselect is more impressive in interviews but has a slim tail risk of $O(n^2)$ if the RNG is really unlucky.

9. Classic Problem 2 — Shuffle an Array

LeetCode 384. Given an array, implement two operations: reset (restore original) and shuffle (return a random permutation). Direct Fisher-Yates application.

```

class Solution {
    int[] original;
    int[] arr;
    Random rand = new Random();

    public Solution(int[] nums) {
        original = nums.clone();
        arr = nums;
    }

    public int[] reset() {
        arr = original.clone();
        return arr;
    }

    public int[] shuffle() {
        int[] shuffled = arr.clone();
        for (int i = shuffled.length - 1; i > 0; i--) {
            int j = rand.nextInt(i + 1);
            int tmp = shuffled[i];
            shuffled[i] = shuffled[j];
            shuffled[j] = tmp;
        }
    }
}

```



```

    }
    return shuffled;
}
}

```

10. Classic Problem 3 — Linked List Random Node

LeetCode 382. Given a linked list of unknown length, return a random node's value in $O(1)$ space. Reservoir sampling shines — you can't know length upfront without scanning, but reservoir sampling picks fairly in a single pass.

```

class Solution {
    ListNode head;
    Random rand = new Random();

    public Solution(ListNode head) {
        this.head = head;
    }

    public int getRandom() {
        int result = -1, count = 0;
        for (ListNode node = head; node != null; node = node.next) {
            count++;
            if (rand.nextInt(count) == 0) result = node.val;
        }
        return result;
    }
}

// Time per query:  $O(n)$ 
// Space:  $O(1)$ 

```

Note the trade-off — each `getRandom()` call scans the whole list. If queries are frequent, storing all values in an array ($O(n)$ space) is faster per query. Reservoir sampling wins when memory is tight.

11. Classic Problem 4 — Random Pick Index with Duplicates

LeetCode 398. Given an array with duplicates and a target value, return a UNIFORMLY RANDOM index that equals target. Reservoir sampling — treat matching indices as a stream.

```

class Solution {
    int[] nums;
    Random rand = new Random();

    public Solution(int[] nums) {
        this.nums = nums;
    }

    public int pick(int target) {

```

```

        int result = -1, count = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == target) {
                count++;
                if (rand.nextInt(count) == 0) result = i;
            }
        }
        return result;
    }
}

```

Beautifully simple — same reservoir pattern, but filtered to matching values. No extra storage needed; each pick is a single pass.

12. Classic Problem 5 — Random Pick with Weight

LeetCode 528. Given an array of weights, `pickIndex()` should return index `i` with probability `weights[i] / sum`. Prefix sum + binary search — exactly the algorithm from Section 7.

This is a very common Google interview question — worth memorizing. It combines prefix sums (Ch 40), binary search (Ch 43), and weighted probability into one elegant $O(\log n)$ query.

13. Classic Problem 6 — Random Point in Non-Overlapping Rectangles

LeetCode 497. Given a list of rectangles, pick a random point uniformly from their combined area. Use weighted sampling on rectangles (weight = area), then a uniform pick within the chosen rectangle.

```

class Solution {
    int[][] rects;
    int[] prefixAreas;
    int totalArea = 0;
    Random rand = new Random();

    public Solution(int[][] rects) {
        this.rects = rects;
        prefixAreas = new int[rects.length];
        for (int i = 0; i < rects.length; i++) {
            int[] r = rects[i];
            int area = (r[2] - r[0] + 1) * (r[3] - r[1] + 1);
            totalArea += area;
            prefixAreas[i] = totalArea;
        }
    }

    public int[] pick() {
        int target = rand.nextInt(totalArea) + 1;
        int lo = 0, hi = rects.length - 1;
    }
}

```

```

        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (prefixAreas[mid] < target) lo = mid + 1;
            else hi = mid;
        }
        int[] r = rects[lo];
        int x = r[0] + rand.nextInt(r[2] - r[0] + 1);
        int y = r[1] + rand.nextInt(r[3] - r[1] + 1);
        return new int[]{x, y};
    }
}

```

14. Random Number Generation Tools in Java

Java API	Purpose
Math.random()	Returns double in [0.0, 1.0). Simple, thread-safe.
Random rand = new Random();	Full-featured RNG. rand.nextInt(n) → int in [0, n).
rand.nextInt(n)	Uniform int in [0, n).
rand.nextInt(hi - lo + 1) + lo	Uniform int in [lo, hi].
ThreadLocalRandom.current()	Faster for multi-threaded code. Same API as Random.
SplittableRandom	For parallel streams. Very high performance.
Seeding	new Random(seed) for reproducibility (tests, debugging).

For interviews: Math.random() and new Random() are always fine. If asked about performance or concurrency, mention ThreadLocalRandom. Never write your own RNG — Java's PRNGs are well-tested (and getting them wrong destroys uniformity guarantees).

15. When to Reach for Randomization

If you need to...	Use
Kth smallest / largest without full sort	Quickselect
Sample K uniform items from a stream	Reservoir sampling
Sample 1 uniform item from a linked list	Reservoir sampling (1-element case)

If you need to...	Use
Random pick weighted by probability	Prefix sums + binary search
Uniform random permutation	Fisher-Yates shuffle
Fast probabilistic primality test	Miller-Rabin (Monte Carlo)
Fast approximate distinct count	HyperLogLog
Sample point in weighted regions	Weighted sampling + local uniform
Anti-adversarial hashing	Randomized hash seeds
Sample 1 element from a large group by key	Reservoir per key or bucketed sampling

16. Complexity Summary

Algorithm	Time	Space
Quickselect (avg)	$O(n)$	$O(1)$ iterative
Quickselect (worst)	$O(n^2)$	$O(1)$
Reservoir sampling (1 item)	$O(n)$ total, $O(1)$ per element	$O(1)$
Reservoir sampling (K items)	$O(n)$	$O(k)$
Fisher-Yates shuffle	$O(n)$	$O(1)$
Weighted random pick	$O(n)$ build, $O(\log n)$ per query	$O(n)$

17. Common Mistakes

Mistake 1: Wrong random range for Fisher-Yates

```
// WRONG – picks from entire array each iteration → biased distribution
for (int i = 0; i < arr.length; i++) {
    int j = rand.nextInt(arr.length);
    swap(arr, i, j);
}

// RIGHT – restrict to [0, i] (or [i, n-1]) for uniform permutation
for (int i = arr.length - 1; i > 0; i--) {
    int j = rand.nextInt(i + 1);
    swap(arr, i, j);
}
```

Mistake 2: Off-by-one in reservoir sampling

```
// WRONG – counter starts at 0, but nextInt(0) throws
if (rand.nextInt(count) == 0) pick = value;

// FIX – always increment count FIRST, so nextInt() gets a positive
argument
```

Mistake 3: Skipping randomized pivot in Quickselect

Using `arr[hi]` as the pivot every time makes sorted input a worst-case $O(n^2)$ scenario. Always pick a random pivot BEFORE partitioning.

Mistake 4: Comparing to 0 instead of a range in weighted pick

```
// WRONG – nextInt(total) is in [0, total), but we want [1, total]
int target = rand.nextInt(total); // x subtle: at 0, no prefix ≥ 0
exists cleanly

// SAFE – use [1, total] and search for first prefix ≥ target
int target = rand.nextInt(total) + 1;
// (Or use [0, total-1] and search for first prefix > target – both work.)
```

Mistake 5: Not testing distribution

Randomized algorithms compile and run — but might produce a biased distribution. Test by running many trials and checking that outputs are approximately uniform. Bugs here are silent.

Mistake 6: Using `Math.random() * n` and casting

```
int r = (int)(Math.random() * n); // ✓ OK, but subtle floating-point
rounding
int r = rand.nextInt(n);          // ✓ safer, integer arithmetic only
```

Mistake 7: Recreating `Random()` every call

```
public int pick() {
    Random r = new Random();          // x inefficient, potentially non-
    random
    return r.nextInt(n);
}

// FIX: create ONE Random instance and reuse it
Random rand = new Random();          // as a field
```

18. Best Practices

- Use ONE `Random` instance per class, not one per call
- For Fisher-Yates, restrict the random index to `[0, i]` — never full array
- For Quickselect, ALWAYS use a random pivot to prevent worst-case $O(n^2)$

- Reservoir sampling: increment counter BEFORE the nextInt() check
- For weighted pick, use prefix sums + binary search — $O(\log n)$ per query
- For reproducible tests, seed the RNG: new Random(42)
- Say "expected $O(n)$ " not " $O(n)$ " for Quickselect — worst case exists
- Test distributions empirically — bugs in randomized code are silent

19. Quick Reference

Concept	Meaning
Randomized algorithm	Uses random numbers to make choices
Las Vegas	Always correct, random runtime
Monte Carlo	Fixed runtime, may be wrong with small probability
Quickselect	Find kth smallest in $O(n)$ average
Reservoir Sampling	Uniform sample from a stream in $O(1)$ space
Fisher-Yates	Uniform random permutation in $O(n)$
Weighted pick	Prefix sums + binary search, $O(\log n)$ per query
1/i replacement rule	Replace pick with prob $1/i$ to get uniform
Random pivot	Prevents adversarial worst-case in Quickselect
ThreadLocalRandom	Faster RNG for concurrent code
Seed	For reproducibility in tests and debugging

20. Practice Problems

Try in order — foundational first, then reservoir sampling, then weighted picking.

Quickselect

4. Kth Largest Element in an Array (LeetCode 215) — implement with Quickselect.
5. Wiggle Sort II (LeetCode 324) — Quickselect + partition.
6. Top K Frequent Elements (LeetCode 347) — bucket sort or heap or Quickselect.

Reservoir Sampling

7. Linked List Random Node (LeetCode 382).
8. Random Pick Index (LeetCode 398).

9. Design an Online Random Sample — K items from a stream.

Shuffle & Permutation

10. Shuffle an Array (LeetCode 384).
11. Design Snake Game (LeetCode 353) — random food placement.
12. Insert Delete GetRandom O(1) (LeetCode 380).
13. Insert Delete GetRandom O(1) - Duplicates allowed (LeetCode 381).

Weighted / Probabilistic Pick

14. Random Pick with Weight (LeetCode 528).
15. Random Point in Non-Overlapping Rectangles (LeetCode 497).
16. Generate Random Point in a Circle (LeetCode 478).
17. Implement Rand10() Using Rand7() (LeetCode 470).
18. Random Flip Matrix (LeetCode 519).

Advanced

19. Fibonacci-based random tree generator.
20. Random Number Generator that Follows a Given Distribution.
21. Approximate Distinct Count using HyperLogLog (design-oriented).

21. Final Takeaway

Randomized algorithms are a compact but powerful category. Three ideas — Quickselect, Reservoir Sampling, and Fisher-Yates — solve a huge slice of interview problems that would otherwise require heavier machinery. And the intuition (random pivot avoids adversarial worst cases; $1/i$ replacement gives uniform sampling; restricting random range gives uniform permutation) generalizes to advanced applications like load balancing, streaming analytics, and MCMC methods.

For interviews, master three algorithms: Quickselect (for Kth-order-statistic problems), Reservoir Sampling (for streaming and random-selection problems), and Fisher-Yates (for shuffling). Add the weighted-pick pattern (prefix sums + binary search) and you're set for essentially every randomized-algorithm question a Google interview will throw at you.

Key Takeaway: Quickselect finds kth-smallest in $O(n)$ average — use RANDOM pivots to avoid worst-case $O(n^2)$. Reservoir sampling: replace pick with prob $1/i$ after processing i elements → uniform sampling from streams. Fisher-Yates shuffle: swap $arr[i]$ with a random index in $[0, i]$ — uniform permutation. Weighted random pick: prefix sums + binary search. Las Vegas algorithms are always correct with expected runtime; Monte Carlo trades correctness for speed. In Java, use one Random instance per class and `Math.random()` or `nextInt()` for simple cases.

22. What's Next?

With randomized algorithms covered, your classical DSA + core CS + interview-pattern toolkit is essentially COMPLETE. The remaining directions:

- System Design fundamentals — the natural big step for senior interviews (Google's second phase)
- Advanced DP variants — knapsack variants, digit DP, matrix exponentiation, bitmask DP
- Max Flow / Min Cut — Ford-Fulkerson, Edmonds-Karp
- Computational Geometry — convex hull, line sweep, closest pair of points
- Suffix Arrays / Automata — advanced string processing
- Behavioral / Leadership interviews — Googleyness and the STAR method
- Concurrency & Multithreading — the Java side you touched briefly in Ch 28